K L Deemed to be University

(Koneru Lakshmaiah Education Foundation)

GREEN FIELDS, VADDESWARAM · TERM 3, AY 2025-2026 · PROJECT-BASED LEARNING

MID-TERM EXAMINATION · PAPER SET 2

COURSE	DATA STRUCTURES AND ALGORITHMS - 2	**CODE**	25SC1305E
EXAMINATION	Mid-Term Examination	**PAPER**	Set 2
DURATION	90 minutes	**MAX. MARKS**	45
COORDINATOR	Zeelan Basha CMAK	**MODE**	Project-Based Learning

INSTRUCTIONS TO CANDIDATES

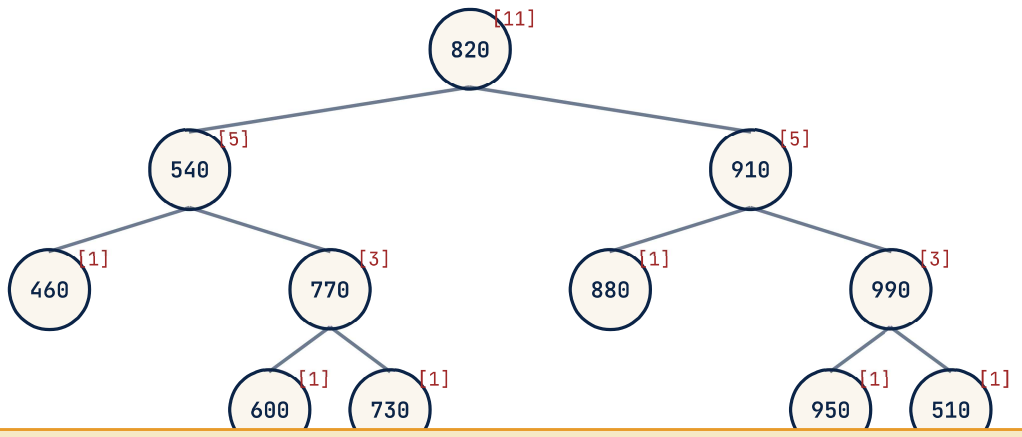
- Answer ALL three questions. Each question carries 15 marks. Maximum: 45 marks.
- Each question is a scenario / case-study / micro-project covering one Course Outcome.
- Show all working, intermediate steps, code listings, diagrams, derivations and reasoning traces.
- Time allowed: 90 minutes. Calculator allowed. No mobile phones, smart watches, or AI assistants.

Q1. CO1 Module 1: BST & AVL Trees (rank-augmented) **[15 Marks]**

SCENARIO / CASE STUDY

QuizClash — live tournament leaderboard. A real-time multiplayer quiz platform tracks **score** for every active player in a self-balancing tree keyed by score (descending). During a flash tournament: 60 000 active players, sustained **10 000 score-update operations per second** (each is a delete-old-score + insert-new-score), and the UI requests *'rank of player X'* and *'top-K players, K ≤ 100'* queries at 2 000 qps. The product team requires every operation — update *and* rank-query — to complete in **O(log n)**. Today's first 11 score updates produce, in order, a tree built from inserts: `820, 540, 910, 770, 880, 460, 990, 600, 730, 950, 510`. Then two updates arrive: player whose score was 540 jumps to 815; player whose score was 910 drops to 685.

Rank-augmented tree — every node carries size(subtree) for O(log n) rank queries




```
Workload: 10 000 update/sec · 2 000 rank-query/sec · 60 000 active players
Required: O(log n) for both update and 'rank of player X' — naive scan 60 000 = too slow
```

SUB	QUESTION	MARKS	CO	BTL
(a)	Construct the AVL tree from the insertion sequence (820, 540, 910, 770, 880, 460, 990, 600, 730, 950, 510), maintaining a **`size(subtree)`** field at every node (the count of nodes in that subtree, including itself). Show the final tree with size fields. Then explain in one or two sentences *why size augmentation enables O(log n) rank queries* in the leaderboard scenario, and why a plain AVL tree (no size fields) cannot meet the 2 000-rank-queries/sec requirement at 60 000 players.	5	CO1	BTL3
(b)	Complete the boilerplate Python function `rank_of(root, key)` that returns the 1-based rank of `key` in the rank-augmented AVL tree (1 = highest score, since the tree is keyed descending). The standard size-update glue is given; you write the rank descent. Then handwrite the rank that your function returns for player score = 770 in your tree from (a).	6	CO1	BTL3
(c)	Apply the two score-update operations from the scenario in order: (i) the player with score 540 changes to 815 (delete 540, insert 815); (ii) the player with score 910 changes to 685 (delete 910, insert 685). For each, show the rotations triggered and the resulting tree. Then **analyse**: how many nodes have their `size` field touched by each update operation, and why is this number *also* O(log n)? Tie this to the throughput requirement (10 000 updates/sec).	4	CO1	BTL4

```
class RankNode {
    int key;
    RankNode left, right;
    int size = 1;
    RankNode(int key) { this.key = key; }
}

static int  size(RankNode n) { return n == null ? 0 : n.size; }
static void updateSize(RankNode n) {
    if (n != null) n.size = 1 + size(n.left) + size(n.right);
}

/** Return 1-based rank of `key`. Tree keys are stored DESCENDING
 *  (root.left.key > root.key > root.right.key). Assume `key` is present. */
static int rankOf(RankNode root, int key) {
    // TODO: walk from root; at each node decide left / right and accumulate rank.
    return 0;
}
```

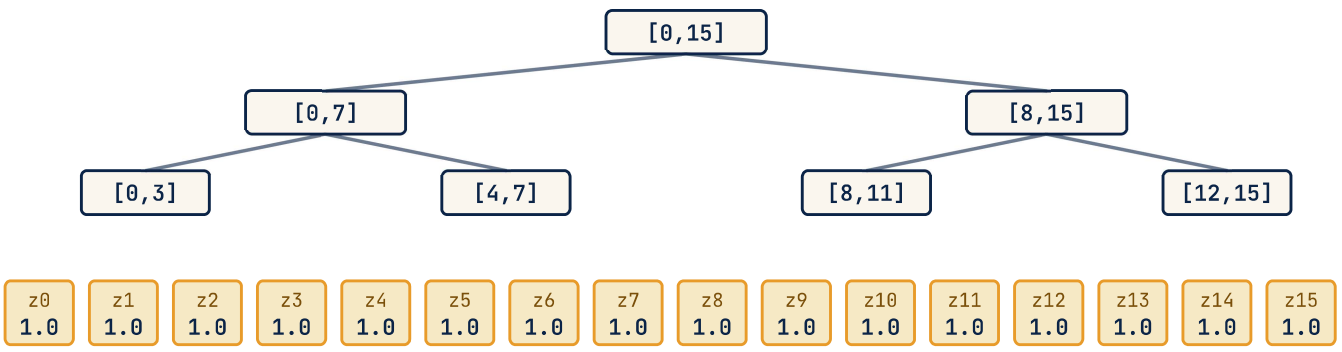
Q2. CO2 | Module 2: Segment Trees with Lazy Propagation [15 Marks]

SCENARIO / CASE STUDY

Uber Bengaluru — surge multiplier on geofenced zone array. The city is divided into **n = 16** geofenced demand zones, indexed 0..15 left-to-right by geographic strip. The system holds a per-zone surge multiplier (a small floating-point value, currently all 1.0). Two operation types arrive every few seconds: *'apply Δ to zones [l, r]'* (add Δ uniformly across a contiguous strip — typical when an event ends or a metro line breaks down) and *'maximum multiplier in zones [l, r]'* (the rider app polls this to decide which adjacent zone offers a cheaper trip). Today's trace, from 7:00 PM to 7:01 PM:

1. `update [3, 9] += 0.5` (M.G. Road event ends → demand drops; this is actually negative for surge, so engineers really apply −0.5; for the sake of this exercise treat the operation as *add* 0.5 to keep all integers ≥ 0)
2. `update [7, 14] += 0.3` (Whitefield IT shift ends)
3. `query max [0, 15]`
4. `update [2, 6] += 0.7` (cricket-stadium emptying)
5. `query max [4, 10]`

Segment tree over 16 zones — supports range add (lazy) + range max



All 16 leaves start at 1.0; updates push range-add into lazy markers on the spanning internal nodes.

SUB	QUESTION	MARKS	CO	BTL
(a)	Trace the segment tree's state after the five operations in the scenario, in order. For each *update*, show which internal nodes receive a **lazy marker** (and the marker's value), and which nodes have their **max** value updated immediately. For each *query*, show the path of internal nodes visited and the answer returned. (Use a compact tabular trace; you do not need to redraw the tree five times.)	6	CO2	BTL3
(b)	Complete the boilerplate `update_range(node, l, r, lo, hi, delta)` below — the lazy-propagation range-add. The push-down helper `push_down` is given; you fill in the recursion + lazy accumulation. Then state the worst-case number of internal nodes visited per operation as a function of n, and plug in `n = 16`.	5	CO2	BTL3
(c)	An engineer proposes replacing the segment tree with a **Fenwick (BIT)**. Argue precisely why a Fenwick tree is the wrong choice *for this workload*: identify which one of the two operation types it cannot handle in O(log n), and describe what would have to change about the workload (or about Fenwick) to make Fenwick adequate. Bonus consideration: why is a *sparse-table* approach also not suitable here?	4	CO2	BTL5

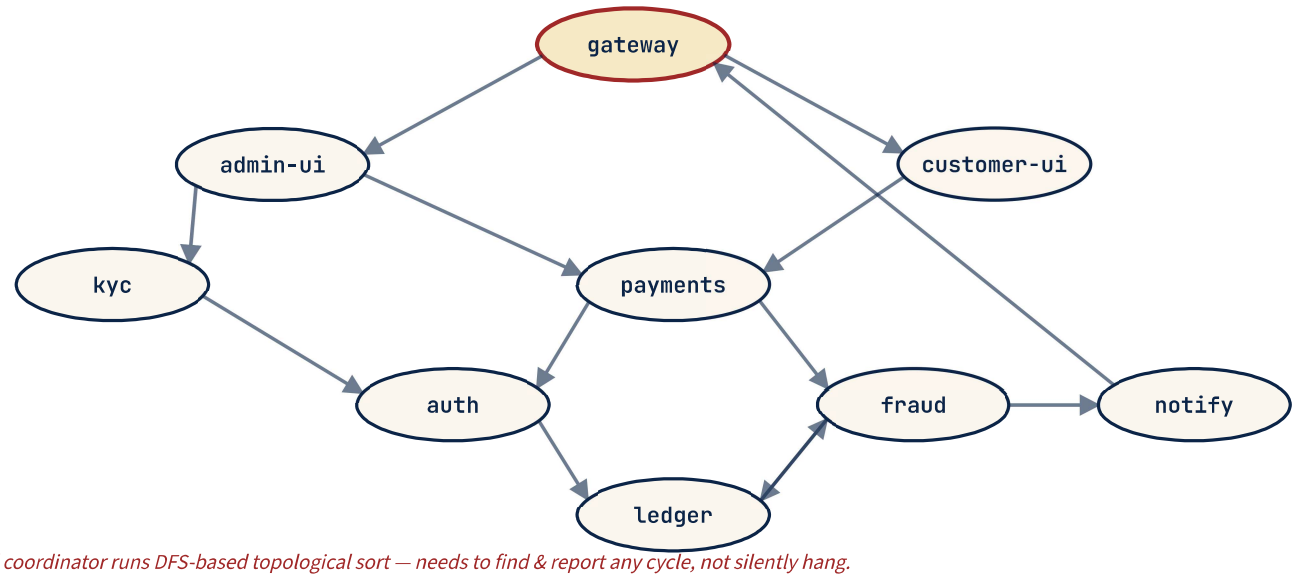
```
class SegTreeLazy {
    long[] tree;       // max value at each node
    long[] lazy;       // pending add for each node
    int n;
    SegTreeLazy(int n) {
        this.n = n;
        tree = new long[4 * n];
        lazy = new long[4 * n];
    }

    void pushDown(int node) {
        if (lazy[node] != 0) {
            tree[2*node]   += lazy[node];
            lazy[2*node]   += lazy[node];
            tree[2*node+1] += lazy[node];
            lazy[2*node+1] += lazy[node];
            lazy[node] = 0;
        }
    }

    /** Add `delta` to every leaf in [l, r]. `lo, hi` is current node's segment. */
    void updateRange(int node, int lo, int hi, int l, int r, long delta) {
        // TODO: handle no-overlap, full-overlap (lazy mark + return), partial-overlap.
    }
}
```

Q3. CO3 | Module 3: DFS, Topological Sort & Cycle Detection **[15 Marks]**

SCENARIO / CASE STUDY

Razorpay backend monorepo — CI build-graph cycle suspected. The Razorpay engineering monorepo has 9 build targets {`auth, payments, kyc, ledger, fraud, notify, admin-ui, customer-ui, gateway`}. Yesterday's CI run hung; the build coordinator (which performs a topological sort of the dependency DAG before scheduling parallel build jobs) failed with a generic 'cycle suspected' message. The build engineer captured the dependency edges from the build manifest (edge `A → B` means "A depends on B; build B before A"): `auth→ledger`, `payments→auth`, `payments→fraud`, `kyc→auth`, `ledger→fraud`, `fraud→notify`, `fraud→ledger`, `admin-ui→payments`, `admin-ui→kyc`, `customer-ui→payments`, `gateway→admin-ui`, `gateway→customer-ui`, `notify→gateway`. Twelve hours of debugging would have been saved by a clear cycle report.

Razorpay build dependency graph (A → B means "A depends on B"); cycle hidden in here



Build coordinator runs DFS-based topological sort — needs to find & report any cycle, not silently hang.

SUB	QUESTION	MARKS	CO	BTL
(a)	Run a **DFS-based topological sort with cycle detection** on the dependency graph shown. Use the colour-marking scheme (WHITE = unvisited, GREY = on current DFS stack, BLACK = finished). Process vertices in alphabetical order on the outer loop. Show the visit-time/finish-time table or a clear DFS trace, and identify the **back-edge(s)** that prove a cycle exists. List the vertices on the cycle in order.	6	CO3	BTL3
(b)	The engineer's draft of `topo_sort` is shown below — it currently reports cycles by raising an exception with no information about *which* vertices are involved. **Modify it** to (i) detect a cycle and (ii) return the list of vertices on the cycle, in cycle order. Boilerplate: WHITE/GREY/BLACK constants and the recursion skeleton are given.	5	CO3	BTL3

```
static final int WHITE = 0, GREY = 1, BLACK = 2;

static class TopoResult {
    List<String> order;       // null if cycle detected
    List<String> cycle;       // empty if acyclic
    TopoResult(List<String> o, List<String> c) { order = o; cycle = c; }
}

static TopoResult topoSortWithCycle(Map<String, List<String>> adj) {
    Map<String, Integer> color  = new HashMap<>();
    Map<String, String>  parent = new HashMap<>();
    List<String> order = new ArrayList<>();   // reverse-topo on success
    List<String> cycle = new ArrayList<>();   // populated if back-edge
    for (String v : adj.keySet()) color.put(v, WHITE);

    for (String u : adj.keySet()) {
        if (color.get(u) == WHITE && dfs(u, adj, color, parent, order, cycle))
            return new TopoResult(null, cycle);   // cycle detected
    }
    Collections.reverse(order);
    return new TopoResult(order, Collections.emptyList());
}

static boolean dfs(String u, Map<String, List<String>> adj,
                   Map<String, Integer> color, Map<String, String> parent,
                   List<String> order, List<String> cycle) {
    color.put(u, GREY);
    for (String v : adj.getOrDefault(u, Collections.emptyList())) {
        if (color.get(v) == WHITE) {
            parent.put(v, u);
            if (dfs(v, adj, color, parent, order, cycle)) return true;
        } else if (color.get(v) == GREY) {
            // TODO: a back-edge found — fill `cycle` with the offending
            //       vertices in correct order, then return true.
        }
    }
    color.put(u, BLACK);
    order.add(u);
    return false;
}
```

SUB	QUESTION	MARKS	CO	BTL
(c)	Once the cycle is identified, the build engineer must **break it with a single edge removal**. Among the edges on the cycle you found in (a), which single edge removal would (i) break the cycle *and* (ii) cause the least disruption to existing modules — i.e., result in the fewest dependent transitive builds becoming stale? Justify by listing, for each candidate removal, how many other targets would be affected. (Use the dependency graph, not the build-time itself.)	4	CO3	BTL5